

HRMS Project Handoff

What's done, how it works, and what to build next

PART 1 — WHAT'S BUILT AND WORKING

1. What This Project Is

This is a self-service extension to the existing Flask HRMS app. Two connected features were built this cycle: a self-service Onboarding wizard (candidate fills their own bank details, documents, and sets a password), and a full Offer Letter → Admin Approval → Candidate E-Sign → HR Countersign → NDA → Onboarding pipeline. Everything is built to run for free — no paid e-sign vendor, no paid email service, no paid PDF service.

The two pieces are connected: when an NDA is fully countersigned, the onboarding invite email fires automatically for that same candidate — nobody has to remember to click "send invite" separately.

2. Status Summary

Area	Status	Notes
Self-service onboarding wizard	Done	Candidate sets password, enters bank + documents via a token link.
Offer creation + HR/Admin approval flow	Done	Draft → Pending Approval → Approved/Changes Requested.
Candidate e-signature (offer)	Done	Typed name + checkbox, IP + timestamp captured as audit trail.
HR countersignature (offer)	Done	HR types name after candidate signs; final PDF regenerated with both blocks.
NDA auto-generation + e-sign + countersign	Done	Fires automatically the moment the offer is countersigned.
Auto-trigger onboarding invite	Done	Fires automatically the moment the NDA is countersigned.
Three offer types (Full Time / Contract / Intern)	Done	Each is now a genuinely different document, matched to the company's real reference docs.
PDF letterhead (logo, watermark, serif font, signature boxes)	Done	Matches the reference "Shivangi" offer letter's structure and spacing.
₹ (Rupee) rendering + Indian digit grouping	Done	Fixed via an embedded Unicode font (DejaVu Serif); numbers read as 5,50,000 not 550,000.
CTC/compensation math consistency	Done	Total row is always computed from components — can no longer contradict itself.
Self-service template editor (HR-facing)	Partial	Edit the HTML of all 4 documents + watermark/logo appearance — works, but currently saves live with no review. Needs the approval-queue retrofit in Part 2, Task 2.
On-screen preview pages match the final PDF styling	Done	offer_review, esign_offer, esign_nda, nda_review all restyled to match.
Extra Employee Info Form fields folded into onboarding	Not started	Family info, education history, certifications, previous employer — deferred, see section 6.
Automated tests	Not started	Everything so far has been verified by hand (rendered PDFs, manual clicks) — no test suite exists yet.

Area	Status	Notes
Template editor — on-screen preview pages	Partial	Custom uploaded logo/watermark only shows in the generated PDF, not yet in the browser preview pages.

3. Architecture & Tech Stack

- Backend: Flask, Python. Blueprints live under `hrms/onboarding/` and `hrms/offers/`.
- Database: Supabase Postgres (project `vcqtbprceutgqcgbpnte`). Every DB-touching function follows a dual-path pattern — try direct `psycopg2` via `utils/db.py` first, fall back to the Supabase REST API (`utils/supabase_rest.py`) on any exception. This means the app keeps working even if the direct DB connection is flaky.
- File storage: Supabase Storage, for generated PDFs (offer letters, NDAs).
- Email: Gmail SMTP via `utils/mailer.py` — every notification (offer sent, awaiting countersign, NDA sent, onboarding invite) is one function in that file.
- PDF generation: `xhtml2pdf` (`pisa.CreatePDF`). Templates use `xhtml2pdf`-specific `@page` / `@frame` CSS to create repeating per-page elements — a full-page watermark frame and a per-page signature-box footer frame.
- Fonts: DejaVu Serif (regular + bold) is embedded directly into every generated PDF as a base64 `@font-face`, because `xhtml2pdf`'s built-in fonts don't include the ₹ glyph. This also gives the documents a serif, Times-like look matching the company's real signed letters.
- E-signature: in-app click-to-accept (typed legal name + checkbox + captured IP + UTC timestamp, baked into the final PDF). This is a valid electronic signature under India's IT Act, 2000 — it is NOT an Aadhaar-eSign or DSC-grade signature (that requires a paid third party).

Key files

```
hrms/offers/routes.py          - offer/NDA pipeline: all routes, PDF rendering,
templates, e-sign logic
hrms/onboarding/routes.py      - self-service onboarding wizard routes
templates/hrms/offers_letterhead.html - the PDF shell (logo, watermark, footer, fonts)
templates/hrms/offer_templates_editor.html - HR-facing template + appearance editor
templates/hrms/offer_review.html, offer_form.html, offers_pipeline.html - HR/Admin screens
templates/esign_offer.html, esign_nds.html - public candidate-facing e-sign pages (token-
gated, no login)
templates/hrms/nda_review.html - HR NDA countersign screen
utils/mailer.py                - all outbound emails
static/fonts/DejaVuSerif*.ttf, static/images/logo_*.png - embedded PDF assets
```

Database tables touched

- `hrms_employees` — existing table; offer flow creates a shell row here (`status='Offer Pending'`) at draft time.
- `employee_offers` — one row per offer letter: candidate + compensation fields, `offer_type`, `status`, `content_html`, `sign_token`, `hr_signed_name/at`, `pdf_url`.
- `employee_ndas` — one row per NDA, mirrors `employee_offers`' signature/status fields.
- `offer_templates` — the 4 editable HTML templates (`offer_letter_fulltime` / `offer_letter_contract` / `offer_letter_intern` / `nda`), each seeded from a Python default the first time it's fetched.
- `company_settings` — extended this cycle with `offer_logo_wordmark_b64`, `offer_watermark_b64`, `offer_watermark_opacity`, `offer_watermark_width_cm`, `offer_logo_width_px` for the appearance editor.

4. Workflows

4.1 Offer → Sign → NDA → Onboarding (the main pipeline)

- 1. HR fills out the offer form, picking an Offer Type — Full Time, Contract, or Intern. This creates a minimal `hrms_employees` row and an `employee_offers` row, `status = Draft`.
- 2. HR submits for approval → `status = Pending Approval`.
- 3. Admin reviews the rendered letter and either Approves → `Approved`, or Requests Changes with a comment → `Changes Requested` (goes back to HR).
- 4. HR sends the approved offer → `status = Sent`. A one-time secure link is emailed to the candidate (token-based, no login required).
- 5. Candidate opens the link, reviews the letter, types their legal name, checks "I Accept" → `status = Signed`. Name, IP, and timestamp are captured. HR is notified by email that it's awaiting countersignature.
- 6. HR opens the offer, types their name, clicks Countersign → `status = Countersigned`. The final PDF (with both signature blocks) is generated and uploaded. This step automatically creates and sends an NDA — no separate approval cycle for the NDA.
- 7. Candidate e-signs the NDA the same way → `status = Signed` (awaiting HR).
- 8. HR countersigns the NDA → `status = Countersigned`. The moment this happens, the onboarding invite email fires automatically for that same employee — nobody has to click "send invite."
- 9. Candidate follows the onboarding link, sets a password, and fills in bank details + documents self-service.

4.2 Editing a document template

- HR/Admin opens Offer Letters → Edit Templates.
- Each of the 4 documents (Full Time offer, Contract Agreement, Internship Agreement, NDA) is a raw-HTML editor with clickable placeholder chips (`{{candidate_name}}`, `{{compensation_table}}`, etc.).
- "Preview PDF" renders the current (even unsaved) edits with sample data in a new tab before anything touches a real offer.
- "Save Changes" writes to the `offer_templates` table immediately — the next offer created uses the new wording. Offers already sent are unaffected (their `content_html` was already baked in at send time).
- "Reset to Default" restores the original wording for that one document.
- A separate Watermark & Logo tab lets HR upload a new header logo / watermark PNG and adjust opacity and size with sliders — applies to every document's PDF from that point on.

5. How to Run / Where Things Live

Start the app: python app.py, from the hrms-main folder. Restart it after any .py file change (template/HTML/text edits inside offer_templates_editor.html apply live, no restart needed — but a change to routes.py needs a restart).

Supabase project: vcqtbprceutgqcgbpnte — ask Kunal for dashboard access if you don't have it yet.

Gmail SMTP creds: in the app's env config (env.local.txt / environment variables) — do not commit these anywhere public.

Legal entity name: set via LEGAL_ENTITY_NAME env var (defaults to "ANTI.AI PRIVATE LIMITED") — separate from the shorter COMPANY_NAME env var used elsewhere in the app.

If offers or NDAs look like they're using old wording after a template edit, check the offer_templates table directly — each row is only seeded once (on first fetch) and won't auto-update if the Python default changes in code. Use the template editor UI instead of touching the DB by hand — it handles this correctly.

6. Known Gaps & Pending Work

Carried over from earlier planning, not started yet

- Fold the extra Employee Information Form fields into the onboarding wizard as optional: family info table, education history, certifications, previous employer + reason for leaving, previous HR contact, father/spouse name, citizenship.

Surfaced during this cycle, not acted on

- Schema-drift audit — some salary/bonus/reimbursement tables may not match what the UI actually writes; worth a pass before this goes into heavier use.
- Hardcoded dashboard metrics — some numbers on the main dashboard are static rather than computed live.
- No forgot-password flow for HRMS logins.
- No bulk actions (e.g. bulk-approve offers, bulk-remind candidates who haven't signed yet).
- No audit-trail viewer — signature audit data (name/IP/timestamp) is captured and baked into the PDF, but there's no screen to browse it across all offers.
- No holiday calendar feature.
- Template editor's custom logo/watermark only reflects in the generated PDF — the on-screen preview pages still show the default bundled images, not a custom upload.
- No automated tests — everything has been verified by rendering PDFs and manually clicking through the flow. Worth writing at least a smoke-test suite before this goes further.

7. The Problem This Solves

Today the HRMS only sends email when the system decides to — offer sent, NDA signed, onboarding invite. HR never composes anything themselves inside the app, so the moment they need to schedule an interview, message a candidate ad hoc, or announce something to the team, they leave the app and go to Gmail. That round-trip is the thing to close.

The goal isn't "add an email feature" — it's that HR should never have a reason to open Gmail or a calendar app for anything candidate- or employee-facing. Email and calendar become plumbing underneath the HRMS, not a separate tool HR has to remember to use.

8. Confirmed Scope

- Calendar invites: build the free .ics-attachment approach first, then add real Google Calendar sync — both are free, so both are in scope, just sequenced (Task 6 depends on Task 3 proving itself).
- Sending account: personal Gmail (not Google Workspace) — hard cap of roughly 500 sends/day. This shapes the bulk-announcement design (Task 4).
- Sensitive HR actions require Admin approval first: document template edits, watermark/logo changes, deleting records, company settings changes, and bulk announcement sends. This is a generalized approval queue, not a one-off check bolted onto each feature — see Task 2.
- That approval requirement is retroactive, not just for new features — the offer/candidate delete action that already ships today gets gated the same way as part of Task 2.
- Both HR and Admin get a notification inbox, with different content in each — HR sees things like "candidate signed, needs your countersign"; Admin sees things like "offer pending approval," "template edit awaiting your review," "SLA aging." See Task 1.
- All sixteen tasks below are in scope for this build — you'll be working through all of them, not just a subset; this document sequences them so each is useful on its own and lays groundwork for the next.
- Candidate replies stay in the candidate's own inbox for now — no shared inbox / IMAP thread view in this phase. One-way sending only.

9. Task List (in build order)

Each task is independently useful on its own, and later tasks reuse what earlier tasks build — the notification inbox and approval queue in particular are infrastructure everything else should plug into from day one, rather than each feature reinventing its own version.

Task	What ships	Why this order
Task 1	In-app notification inbox (bell icon + feed) — HR and Admin, different content	Cheapest to build, and every later task (interview accepted, announcement sent, approval requested) should emit into it from day one instead of being retrofitted later. Admin's inbox is also where Task 2's approval requests surface.
Task 2	Sensitive-action approval queue (template edits, appearance changes, deletes, company settings, bulk sends)	A security/governance layer, not a feature HR asked for directly — but it changes how several already-built and soon-to-be-built actions work, so it belongs early, right after notifications exist to carry it.
Task 3	Meeting/interview invites via .ics email attachment	The most directly requested feature, zero new infrastructure (reuses the existing Gmail SMTP sender), and immediately removes one whole reason HR opens a separate calendar app.
Task 4	Announcements / bulk message composer + template library	Reuses the exact pattern already shipped for offer letter templates (HR already knows that UI), plus a send-queue that respects the personal-Gmail 500/day cap and routes through Task 2's approval queue.
Task 5	Pre-offer candidate pipeline (mini-ATS)	The biggest scope of the six — a real "applied → screening → interview → offer" board. Do this after Tasks 1–4 so interview invites, notifications, and the approval queue already exist to plug into it, rather than building them twice.
Task 6	Real Google Calendar sync (OAuth)	Optional upgrade once Task 3's .ics version has proven useful. Needs a Google Cloud project + OAuth consent screen — real setup work, not just code — so it's sequenced after Task 3, not because it's low-value.
Task 7	Employee self-service — leave, payslips, personal documents	Gives the Employee login its first real reason to be used day-to-day; payslip access needs a quick schema-drift check first (see Part 1's Known Gaps).
Task 8	Company directory, announcements feed, holiday calendar	Three small read-only Employee views; the announcements feed is a natural byproduct once Task 4 exists.
Task 9	Employee help desk (query tickets) + policy acknowledgment	Query tickets are new but simple; policy acknowledgment reuses the existing e-sign infrastructure almost as-is.
Task 10	Offboarding workflow	Mirrors the onboarding pattern that already exists — exit interview, asset return, final settlement, access revocation.
Task 11	HR operational extras — leave approval, lifecycle reminders, bulk onboarding, duplicate detection	Grouped because each is individually small; together they close the remaining day-to-day HR friction points.

Task	What ships	Why this order
Task 12	Admin control center — user/role management, company settings, audit log	Turns Admin from "HR with an approve button" into a real admin surface; the base Task 15 later builds on.
Task 13	Usage/quota dashboard + HR analytics dashboard	Operational visibility into Gmail/Storage limits, plus a hiring-funnel view once Task 5's pipeline has real data in it.
Task 14	Data retention & deletion policy for rejected-candidate PII	A compliance safety net — auto-purges old rejected-candidate PII on a schedule instead of it sitting around indefinitely.
Task 15	Granular role permissions beyond HR/Admin	Only worth doing once real usage shows which permissions actually need splitting apart — sequenced after the rest of the HR/Admin work.
Task 16	Mobile-friendly / PWA pass	Touches every screen built in Tasks 1–15, so it's sequenced last on purpose rather than being redone repeatedly as new screens ship.

Task 1 — Notification Inbox (HR and Admin, different content)

A bell icon in the top nav (base.html) with an unread badge and a dropdown feed — on both the HR and Admin logins, but fed from different event types. Every existing `mailer.send_*`() call in the codebase should fire a matching in-app notification alongside the email, not instead of it.

What HR sees

- "Candidate signed the offer — awaiting your countersignature"
- "NDA signed — awaiting your countersignature"
- "Your template edit was approved" / "Your template edit was rejected — see comment" (once Task 2 exists)
- "Onboarding completed for [candidate]"

What Admin sees

- "Offer pending your approval" (already exists as an email today — this adds the in-app version)
- "Template edit awaiting your review" / "Delete request awaiting your review" / "Bulk send awaiting your review" (once Task 2 exists)
- Aging alerts: "3 offers have been pending approval for 5+ days", "2 approval-queue items have been waiting 3+ days"

notifications (new table)

Column	Type	Notes
id	uuid	PK
recipient_role	text	'HR' or 'Admin' — determines which login sees it. Notifications are role-scoped, not per-user, since each side is a small shared team right now
type	text	e.g. 'offer_signed', 'nda_countersigned', 'approval_requested', 'approval_resolved', 'interview_accepted', 'announcement_sent'
message	text	short human-readable line, e.g. "Jordan Sample signed their offer letter"
link	text	in-app URL to jump to the relevant record
read_at	timestampz, nullable	null = unread
created_at	timestampz	default now()

Implementation note: wrap the existing mailer calls — every place the code currently does `mailer.send_something(...)`, add one line to also insert a notifications row with the right `recipient_role`. This is a mechanical pass through `hrms/offers/routes.py` and `hrms/onboarding/routes.py`, not new architecture. Build this before Task 2 so the approval queue has somewhere to surface its requests from day one.

Task 2 — Sensitive-Action Approval Queue

The security layer requested directly: any HR action that changes shared, sensitive, or destructive state doesn't apply immediately — it's queued as a pending change, Admin sees it in their notification inbox, and it only takes effect once Admin approves it. Reject sends it back to HR with a comment, the same pattern already proven for offer "Changes Requested."

Why one generalized queue, not a check bolted onto each feature

Template edits, appearance changes, deletes, company settings, and bulk sends all need the same shape of workflow: HR proposes → Admin reviews a diff → Admin approves or rejects. Building that once as a reusable table and a pair of UI patterns (a "pending changes" list for Admin, a "pending your approval" state for HR) means every future sensitive action gets approval gating for free by writing into this queue, instead of every feature re-inventing its own review flow.

admin_approval_queue (new table)

Column	Type	Notes
id	uuid	PK
action_type	text	'template_edit' / 'appearance_change' / 'delete_offer' / 'delete_candidate' / 'company_settings_change' / 'bulk_send'
target_table	text, nullable	which table this affects, e.g. 'offer_templates', 'employee_offers'
target_id	uuid, nullable	the specific row, where one applies — null for things like a bulk send that don't map to a single record
payload_before	jsonb, nullable	snapshot of current state, for edits/deletes — this is what powers the diff view Admin sees
payload_after	jsonb	the proposed new state, or the details of the action being requested
requested_by	text	which HR user requested it
status	text	Pending / Approved / Rejected
admin_comment	text, nullable	required on reject, same UX as offer "Changes Requested"
resolved_by	text, nullable	
resolved_at	timestampz, nullable	
created_at	timestampz	default now()

How this changes existing routes

- `save_template()` and the appearance-save routes in `hrms/offers/routes.py` currently write directly to `offer_templates` / `company_settings`. They change to insert an `admin_approval_queue` row instead, and return "submitted for approval" to the HR UI rather than "saved."
- `delete_offer()` (already shipped today) changes the same way — instead of deleting immediately, it queues a `delete_offer` request. This is the retroactive part: the one exception that exists today gets closed, not left as a special case.
- A new Admin screen lists pending queue items with a before/after diff (for edits, a simple side-by-side of `payload_before` vs `payload_after`; for deletes, just what's about to be removed) and Approve/Reject buttons. Approve actually executes the underlying write that used to happen synchronously; reject stores the comment and notifies HR.
- Every insert into this table should also insert a matching Admin notification (Task 1) — that's the whole point of sequencing Task 1 first.

Task 3 — Meeting / Interview Invites (.ics)

Why .ics-over-SMTP first, not the Google Calendar API

Attaching a real calendar file (.ics) to the email we already send via Gmail SMTP gives the candidate native Accept/Decline buttons in Gmail, Outlook, and Apple Mail — no Google account linking, no OAuth, no new infrastructure. It's the same underlying calendar standard Google Calendar itself uses, just delivered by email instead of API.

How it works technically

- Use Python's icalendar library (pure Python, free, no external service) to build a VEVENT — needs a stable UID (generate one per interview, store it), DTSTART/DTEND, SUMMARY, LOCATION or a video link, ORGANIZER, and ATTENDEE lines.
- Attach the .ics to the email with Content-Type: text/calendar; method=REQUEST — this is what makes email clients render it as an actual invite instead of a plain file attachment.
- Store the UID and a SEQUENCE number against the interview record. Rescheduling means re-sending the same UID with SEQUENCE incremented by 1 — that's what makes the candidate's calendar app correctly update the existing event instead of creating a duplicate. Cancelling means sending METHOD:CANCEL with the same UID.
- Add a small new mailer.py function, e.g. send_meeting_invite(to_email, subject, start, end, location, ics_bytes) — follows the same pattern as every other function already in that file.

candidate_interviews (new table)

Column	Type	Notes
id	uuid	PK
employee_id	uuid	FK -> hrms_employees (or the future candidates table from Task 5)
scheduled_at	timestampz	
duration_minutes	int	default 30 or 45
location	text	physical address or video call link
ics_uid	text	stable UID for this event, used for reschedule/cancel
ics_sequence	int	default 0, increment on every reschedule
status	text	Scheduled / Rescheduled / Cancelled / Completed
scheduled_by	text	HR user who booked it
created_at	timestampz	default now()

UI: a "Schedule Interview" button on a candidate's record (offer_review.html today, or the new candidate pipeline once Task 5 exists) opens a small form — date, time, duration, location/link — and sends the invite on submit. A simple list view shows upcoming interviews so HR can see their week without opening a calendar app at all.

Task 4 — Announcements & Bulk Composer

A "write and send" screen, deliberately built on the same pattern as the offer-letter template editor, so HR already knows the UI: pick or write a message, pick recipients, preview, send.

Recipient picker

- A single candidate/employee (by email)
- A department (filtered from hrms_employees)
- Everyone active (all employees with status = 'Active')
- A custom pasted list of emails, for one-off cases outside the system

Respecting the personal-Gmail 500/day cap

Because the sending account is personal Gmail, a bulk send needs a queue, not a synchronous loop that fires 200 emails in one request. The UI should show "this will use N of your ~500 remaining sends today" before confirming, and refuse (or offer to schedule the rest for tomorrow) if a send would exceed the daily budget.

Approval gating

A send to a single recipient goes out immediately, same as today's transactional emails. A send to a department or everyone active is a bulk send — per the confirmed scope, that routes through Task 2's approval queue first (action_type = 'bulk_send', payload_after holding the recipient count, subject, and body) rather than firing the moment HR clicks send.

outbound_messages (new table)

Column	Type	Notes
id	uuid	PK
subject	text	
body_html	text	same {{placeholder}} pattern as offer_templates, e.g. {{candidate_name}}
recipient_email	text	one row per recipient, even for a bulk send — makes retry/status tracking trivial
status	text	Queued / Sent / Failed
created_by	text	
sent_at	timestampz, nullable	

Sending: a small worker loop (APScheduler, still free, already common in Flask apps — no new paid infrastructure) that pulls Queued rows a few at a time, e.g. every 30–60 seconds, respecting a per-minute throttle so Gmail doesn't flag the account for spam-like sending behavior. A saved message_templates table (identical shape to offer_templates) covers the "announcement templates" half of this — e.g. "Interview Reminder," "Welcome Email," "Policy Update."

Task 5 — Pre-Offer Candidate Pipeline

Today the pipeline starts at "offer letter." There's no tracking for a candidate before that — applied, screened, interviewed. If HR is going to live in this app, that's the gap that will surface fastest once Tasks 1–4 are in daily use.

candidates (new table)

Column	Type	Notes
id	uuid	PK
full_name	text	
email	text	
phone	text, nullable	
role_applied	text	
source	text	e.g. referral, job board, LinkedIn
resume_url	text, nullable	uploaded to Supabase Storage, same pattern as offer PDFs
stage	text	Applied / Screening / Interview Scheduled / Interview Completed / Offer Extended / Rejected
owner	text	which HR person is driving this candidate
notes	text	freeform
created_at	timestampz	default now()

UI: a simple kanban-style board grouped by stage (drag-and-drop is a nice-to-have, not required for v1 — a dropdown to move stage is enough). A "Move to Offer" action on a candidate at the Offer Extended stage pre-fills the existing offer_form.html with their name/email/role, handing off cleanly into the pipeline that already exists. Interview invites from Task 3 attach to a candidates.id here once this table exists, instead of only to an existing employee record. Deleting a candidate from this board is itself a sensitive action per Task 2 — it queues a delete_candidate request rather than removing the row immediately.

Task 6 — Real Google Calendar Sync (optional upgrade)

Once the .ics version has been used for a while and proven the value, this task makes interview events actually appear on HR's own Google Calendar and syncs RSVPs back automatically — Google sends the invite itself instead of us hand-building the email.

What this actually requires (setup work, not just code)

- A Google Cloud project (free) with the Calendar API enabled.
- An OAuth consent screen. Because the account is a personal Gmail (not Workspace), the app type is "External" — for a small team, keep it in Testing mode and add each HR user as a test user (up to 100 allowed) to avoid needing Google's app-verification review, which is a slower process meant for public-facing apps.
- The Google account itself is already settled — this is the same single, shared source Gmail account the app already sends and saves everything through today. Two to three HR team members already have access to it, and Kunal manages/holds access to it, so there's no new "whose calendar is this" decision to make.
- google-api-python-client + a stored OAuth refresh token (encrypted at rest, not just a plain DB column) so the app can create events without a human re-authing each time.
- `events.insert(..., sendUpdates='all')` creates the event and emails attendees automatically — replaces the manual .ics-building from Task 3, which can stay as a fallback if this integration is ever unavailable.

This is genuinely free (Calendar API has no cost at this volume). The one thing that still needs a human step before writing code is the Google Cloud project itself — reach out to Kunal directly when you're ready to start this task, since he holds access to the shared account and will set up (or grant access to) the project.

10. Risks & Things to Watch

- Personal Gmail's ~500/day send cap is real — the queue and the "you're about to use N of your remaining sends" UI in Task 4 aren't optional polish, they're what stops the account from getting silently rate-limited or flagged.
- .ics UID/SEQUENCE handling has to be correct from day one — get this wrong and rescheduling an interview creates a duplicate event on the candidate's calendar instead of updating the existing one, which looks unprofessional.
- The app is currently request-driven only (no background worker) — Task 4's send queue is the first thing that needs a lightweight scheduler (APScheduler is the natural fit, still free, no new paid infrastructure).
- The approval queue (Task 2) is only as good as how fast Admin responds to it — if it becomes a bottleneck (HR waiting days for a template fix to be approved), that's a process problem, not a reason to bypass it. The aging alerts in Task 1 exist specifically to keep the queue from silently backing up.
- If Task 6 happens, the OAuth refresh token is a real credential — store it encrypted, not as a plain column, and treat it with the same care as the Gmail SMTP password.
- None of this needs a paid tier at the volumes this team is at — but if the HR team grows meaningfully, or bulk announcements become a daily habit rather than occasional, personal Gmail's cap will eventually force a move to Google Workspace (paid) or a transactional email provider's free tier — worth flagging now so it isn't a surprise later.
- Task 7's payslip access is the one place where building against the wrong assumption does real damage — showing an employee an incorrect compensation figure is worse than the feature shipping late. Don't start that screen until the schema-drift audit confirms where salary data actually lives.
- Task 14's PII purge is a one-way, destructive scheduled job — test it thoroughly against a copy of the data first, and make sure the retention window defaults to something conservative (e.g. 12+ months) rather than something aggressive, since there's no undo once records are purged.
- Task 15's permission model is easy to over-build — resist designing a fully general permissions system up front. Start from the specific splits that real usage in Tasks 1–14 actually surfaces, not a hypothetical full matrix.

11. What's Needed Before Task 6 Specifically

- Which account this posts to is already answered — the existing shared source Gmail account, the same one 2-3 HR folks already use and that Kunal manages.
- Reach out to Kunal directly when you're ready to start this task — he'll create (or give you access to) the Google Cloud project under that account. This takes a few minutes on his end, so it's not a blocker if you flag it a little ahead of when you actually need it.

Tasks 1–5 and Tasks 7–16 need no new external accounts or setup beyond what already exists — they can start immediately.

Task 7 — Employee Self-Service Expansion (Leave, Payslips & Personal Documents)

This is the thinnest part of the system today — onboarding gets someone in the door, and then there's nothing further for a regular employee to do inside the app. This task gives Employee logins an actual reason to come back.

What ships

- Leave request + balance — an employee can see their current balance and apply for leave; HR approves it (the approval queue this creates is covered under Task 11's leave-approval queue).
- Payslip and compensation history access — a read-only view of past payslips and CTC history.
- A personal document center — their own ID proofs, certificates, and signed offer/NDA copies, pulled from the same Supabase Storage bucket already used for signed documents today.

leave_requests (new table)

Column	Type	Notes
id	uuid	PK
employee_id	uuid	FK to the employee record
leave_type	text	e.g. 'Casual', 'Sick', 'Earned'
start_date	date	
end_date	date	
reason	text, nullable	
status	text	Pending / Approved / Rejected
created_at	timestampz	default now()

Payslip access depends on where salary data actually lives in the app today — flagged in Part 1's Known Gaps as needing a schema-drift audit first. Don't build the payslip screen until that audit confirms where the numbers come from; building against a guess here risks showing an employee the wrong figure, which is worse than not shipping the feature yet.

Task 8 — Company Directory, Announcements Feed & Holiday Calendar

Three small, low-risk Employee-facing views that round out the "why would I open this app" case, all read-only and all built on data that either already exists or is a byproduct of Task 4.

- Employee directory / org chart — a simple searchable list (name, role, department, reporting line) pulled from the existing employee records; a visual org-chart layout is a nice-to-have, not required for v1.
- Announcements feed — the read side of Task 4's bulk composer. Every announcement HR sends already has a row somewhere once Task 4 ships; this is just a filtered, chronological view of "announcements addressed to me" on the Employee login.
- Holiday calendar view — a simple list/calendar of company holidays for the year, maintained by HR (a plain admin-editable table is enough, no need for a dedicated UI beyond a simple form).

Task 9 — Employee Help Desk (Query Tickets) & Policy Acknowledgment

Two features that give employees a way to interact with HR through the app instead of a hallway conversation or a stray email, plus a compliance need (policy sign-off) that's nearly free given what's already built.

Query tickets

- A simple form — subject, description, optional category — that creates a ticket HR can see and respond to inside the app. This is intentionally not a full help desk (no SLAs, no routing rules) — just a structured way to ask HR something and get a threaded reply.

employee_queries (new table)

Column	Type	Notes
id	uuid	PK
employee_id	uuid	FK
subject	text	
description	text	
status	text	Open / In Progress / Resolved
hr_response	text, nullable	
created_at	timestampz	default now()
resolved_at	timestampz, nullable	

Policy acknowledgment

E.g. POSH training sign-off or a code-of-conduct read-and-sign. This reuses the exact e-sign infrastructure already built for offers and NDAs, just pointed at a different document type — the signature capture, PDF rendering, and audit trail all already exist; this is mostly a new document_type value and a UI entry point on the Employee login rather than new plumbing.

Task 10 — Offboarding Workflow

An offboarding workflow mirroring onboarding — exit interview scheduling, asset return tracking, final settlement, and access revocation. The self-service pattern already built for onboarding (token-based wizard, checklist, auto-fired final letter) largely carries over here rather than needing to be designed from scratch.

- Exit interview scheduling reuses Task 3's interview-invite infrastructure — same .ics mechanism, different context.
- Asset return tracking and final settlement are a simple checklist + amount field on the employee's record — no new infrastructure needed beyond a status enum.
- Access revocation is a manual checklist item for now (e.g. "remove from Gmail group," "disable HRMS login") rather than an automated integration — automating it would mean reaching into whatever systems grant that access, which is out of scope here.

offboarding_cases (new table)

Column	Type	Notes
id	uuid	PK
employee_id	uuid	FK
last_working_day	date	
exit_interview_status	text	Not Scheduled / Scheduled / Completed
asset_return_status	text	Pending / Returned / N/A
final_settlement_status	text	Pending / Processed
access_revoked	boolean	default false
notes	text, nullable	
created_at	timestampz	default now()

Task 11 — HR Operational Extras

A grab-bag of smaller HR-side gaps, grouped together because none is large enough to be its own task, but each closes a real day-to-day friction point.

Leave-approval queue

The HR-facing counterpart to Task 7's leave requests — a simple list of pending requests with Approve/Reject, notifying the employee via Task 1's notification inbox either way.

Lifecycle reminders

Automated notification-inbox entries (Task 1, Admin or HR side depending on the event) for probation periods ending and work anniversaries — a scheduled job (the same APScheduler instance introduced for Task 4's send queue) that checks dates daily and fires a reminder a set number of days in advance.

Bulk onboarding

For when hiring happens in batches (e.g. campus hiring) rather than one candidate at a time — a CSV-upload path that creates multiple candidate/offer records at once instead of HR filling out the same form N times. Each generated offer still goes through the normal Draft → Approval → Send flow; this only batches the creation step.

Duplicate candidate detection

Nothing today stops the same person's email being entered twice across the pipeline. A simple check against candidates.email (once Task 5's pipeline table exists) or existing offer records, surfaced as a warning (not a hard block) when HR adds a new candidate — cheap to add, easy to forget until it causes real confusion.

Task 12 — Admin Control Center

Today Admin is really just "HR with an approve button." This task builds out the rest of what a real Admin surface needs: managing who has access, where company-wide settings live, and a record of who did what.

User / role management

A screen to add or remove HR and Admin logins — today this is presumably done by hand in Supabase directly. This is also the natural home for Task 15's granular permissions once that's needed, so it's worth building this screen with that extension in mind even though v1 only needs HR/Admin.

Company settings page

Legal entity name, address, and letterhead defaults are currently only reachable through the offers template editor, which is an odd place for them to live. This moves them to a proper Admin settings page — and per Task 2, changes here are themselves a sensitive action that goes through the approval queue.

System-wide audit log

Not just the offer/NDA signature trail that exists today — once HR is doing everything in this app, "who did what, when" becomes a real compliance question. The `admin_approval_queue` table from Task 2 already captures a good chunk of this for sensitive actions; this task extends logging to cover login events and non-sensitive-but-notable actions too (e.g. an offer being sent, a candidate being added).

`audit_log` (new table)

Column	Type	Notes
<code>id</code>	<code>uuid</code>	PK
<code>actor</code>	<code>text</code>	which user performed the action
<code>action</code>	<code>text</code>	e.g. 'login', 'offer_sent', 'candidate_added'
<code>target_table</code>	<code>text, nullable</code>	
<code>target_id</code>	<code>uuid, nullable</code>	
<code>details</code>	<code>jsonb, nullable</code>	
<code>created_at</code>	<code>timestampz</code>	default <code>now()</code>

Task 13 — Usage & Quota Dashboard + HR Analytics Dashboard

Two dashboards on the Admin login — one operational (are we about to hit a free-tier limit), one strategic (is the hiring pipeline healthy).

Quota/usage view

How many of today's ~500 daily Gmail sends are used, how much Supabase storage is left. The first time something silently fails because a free-tier cap was hit, someone will want to know why, in the app, not by guessing. This reads counters that Task 4's send queue should already be tracking (sends today, remaining headroom) plus a Supabase Storage usage check.

HR-facing analytics view

Once Task 5's candidate pipeline exists: time-to-hire, offer acceptance rate, and how many candidates are stuck at each stage. The data will already be there from Task 5's candidates table and the existing offers table; a dashboard on top is comparatively cheap and is the kind of thing that makes leadership actually value the tool. Build this after Task 5, not alongside it — it needs real pipeline data flowing through first to be worth looking at.

Task 14 — Data Retention & Deletion Policy (Candidate PII Purge)

A data retention/deletion policy for rejected candidates' PII (resumes, addresses, phone numbers) — even a simple "auto-purge candidates rejected 12+ months ago" job avoids this quietly becoming a compliance liability.

- A scheduled job (again, the same APScheduler instance) that finds candidates in a terminal 'Rejected' state past a configurable retention window and purges their PII fields (keeping a minimal anonymized record for historical pipeline stats, if Task 13's analytics need it — otherwise delete the row outright).
- Because this deletes real candidate data, the purge action itself should log to Task 12's audit log before running, even though it's an automated job rather than an HR-initiated action — so there's a record of what was purged and when if anyone ever asks.
- The retention window (e.g. 12 months) should be a company setting (Task 12), not a hardcoded constant, since this is a policy decision Kunal or Admin should be able to change without a code deploy.

Task 15 — Granular Role Permissions (Beyond HR/Admin)

Fine at the current team size; becomes a real need once there are more than a couple of HR logins with different levels of access — e.g. one HR person who should be able to send announcements but not approve template edits. Task 12's user/role management screen is the natural home for this: instead of a single boolean `is_admin`, move to a small set of named permissions (`approve_sensitive_actions`, `send_announcements`, `manage_users`, etc.) that can be assigned per user.

This is explicitly sequenced last among the HR/Admin-side tasks because it's a real design decision, not just a build — get Tasks 1–14 shipped and in use first, since real usage patterns will make it obvious which permissions actually need to be split apart versus which ones were never worth separating.

Task 16 — Mobile-Friendly / PWA Pass

The current UI is desktop-first, which is fine for HR at a desk, but becomes a real gap the first time someone needs to approve something from a phone — and once Task 2's approval queue is the thing standing between HR and getting work done, an Admin who's away from their desk becomes a bottleneck. This task is a responsive-design pass across the existing screens plus a basic PWA manifest (installable icon, offline-friendly shell) — not a separate mobile app.

Sequenced last deliberately: it touches every screen built in Tasks 1–15, so doing it earlier would mean redoing the same responsive work again each time a new screen ships.

12. Notes For Rishi

Read `hrms/offers/routes.py` top to bottom first — its module docstring walks through the whole offer/NDA flow step by step, and it's the most actively-changed part of the codebase right now.

- The three offer types (Full Time / Contract / Intern) are genuinely different documents, not one shared template with a different compensation clause — they were rebuilt to match the company's actual reference Word docs. If wording needs to change, use the template editor (Offer Letters → Edit Templates) instead of editing `routes.py` directly — and once Task 2 ships, that edit goes through Admin approval before it's live.
- The Total CTC / compensation table math is always computed from the individual components — never type a total directly anywhere in code. This was a real bug earlier (a mismatched total shipped to a real candidate) and the fix was to make the total un-typeable.
- PDF generation is done via `xhtml2pdf`, which is quite limited compared to a real browser renderer — CSS support is partial (no flexbox/grid in the PDF, only tables and `@page/@frame`). If you need to change PDF layout, test by rendering to PNG (`pdftoppm`) and looking at it — don't trust that CSS that works in a browser will render the same way in the PDF.
- For Part 2's tasks, work through them roughly in order (1 → 16) — later tasks assume earlier ones exist. Task 2's approval queue assumes Task 1's notifications exist to carry it; Task 5's interview button assumes Task 3 exists; Task 6 assumes Task 3 has been running long enough to prove itself; Task 11's leave-approval queue and duplicate-candidate check assume Task 7 and Task 5 respectively; Task 12's user/role screen is the base Task 15 builds on; Task 13's HR analytics needs Task 5's pipeline data flowing first; Task 16 deliberately comes last since it touches every screen the earlier tasks build.
- None of Tasks 7–16 are optional ideas to re-scope later — they're in scope the same way Tasks 1–6 are. Task 6 and Task 16 are the two where sequencing matters most (Task 6 needs Task 3 proven out first; Task 16 needs the other screens to exist before there's anything to make responsive), everything else can be tackled in whatever order makes sense as you go, as long as the dependencies above are respected.